PropOS 前端页面规格书 v1.9（Flutter + Admin）

> 版本: v1.9
>
> 日期: 2026-04-14
>
> 变更: 移动端从 uni-app (Vue 3 + wot-design-uni) 全面迁移至 **Flutter (Dart 3 + flutter_bloc + Material 3)**。Admin PC 端 (Vue 3 + Element Plus) 规格不变。
>
> 替代: `PAGE_SPEC_v1.8.md.deprecated`

一、全局导航结构

1.1 Flutter 移动端导航（BottomNavigationBar + go_router）

路由方案: `go_router` + `StatefulShellRoute`（保持各 Tab 页面栈独立）

```
// app_router.dart
StatefulShellRoute.indexedStack(
  builder: (context, state, child) => MainShell(child: child),
  branches: [
    StatefulShellBranch(routes: [GoRoute(path: '/dashboard', ...)]),  // Tab 1
    StatefulShellBranch(routes: [GoRoute(path: '/assets', ...)]),     // Tab 2
    StatefulShellBranch(routes: [GoRoute(path: '/contracts', ...)]),  // Tab 3
    StatefulShellBranch(routes: [GoRoute(path: '/workorders', ...)]), // Tab 4
    StatefulShellBranch(routes: [GoRoute(path: '/finance', ...)]),    // Tab 5
  ],
)
```

主壳体 **Widget**:

```
MainShell
└── Scaffold
    ├── body: child                          // 各 Tab 对应的 Navigator
    └── bottomNavigationBar: NavigationBar    // Material 3 NavigationBar
        ├── NavigationDestination(icon: Icons.dashboard, label: "首页")
        ├── NavigationDestination(icon: Icons.apartment, label: "资产")
        ├── NavigationDestination(icon: Icons.description, label: "合同")
        ├── NavigationDestination(icon: Icons.build, label: "工单")
        └── NavigationDestination(icon: Icons.account_balance, label: "财务")
```

角色可见性: 各 Tab 根据 `AuthCubit.state.role` 控制显隐，不满足权限的 Tab 不渲染对应 `NavigationDestination`。

1.2 Flutter 子页面路由

页面	go_router path	模块	TabBar
登录	`/login`	认证	—
首页	`/dashboard`	概览	✅ Tab 1
NOI 分析	`/dashboard/noi-detail`	概览	—
WALE 分析	`/dashboard/wale-detail`	概览	—
资产总览	`/assets`	资产	✅ Tab 2
楼栋详情	`/assets/buildings/:id`	资产	—
楼层热区图	`/assets/buildings/:bid/floors/:fid`	资产	—
房源详情	`/assets/units/:id`	资产	—
合同管理	`/contracts`	租务	✅ Tab 3
合同详情	`/contracts/:id`	租务	—
财务总览	`/finance`	财务	✅ Tab 5
账单列表	`/finance/invoices`	财务	—
KPI 考核	`/finance/kpi`	KPI	—
催收记录	`/finance/dunning`	财务	—
工单管理	`/workorders`	工单	✅ Tab 4
工单详情	`/workorders/:id`	工单	—
新建工单	`/workorders/new`	工单	—
二房东管理	`/subleases`	二房东	—
二房东详情	`/subleases/:id`	二房东	—
通知中心	`/notifications`	通知	—
审批队列	`/approvals`	审批	—

1.3 Admin PC 端导航（侧边栏 + 顶部栏）

> Admin 端规格与 v1.8 完全一致，以下为简述。

侧边栏 **ElMenu** 菜单树:

一级	二级	路由	权限
首页	—	`/dashboard`	全角色

资产管理	资产总览 / 楼栋管理 / 批量导入	`/assets` / `assets/import`	`assets.read`
租务管理	合同管理 / 租客管理	`/contracts` / `tenants`	`contracts.read`
财务管理	财务概览 / 账单管理 / 费用支出 / 水电抄表 / 营业额申报 / 押金台账 / NOI 预算 / 催收管理	`/finance/*`	`finance.read`
工单管理	工单列表	`/workorders`	`workorders.read`
二房东管理	子租赁列表 / 批量导入	`/subleases` / `subleases/import`	`subleases.read`
KPI 考核	考核看板	`/finance/kpi`	`kpi.read`
通知中心	—	`/notifications`	全角色
审批队列	—	`/approvals`	`approvals.manage`
系统设置	用户管理 / 组织架构 / KPI 方案 / 递增模板 / 预警中心 / 审计日志	`/settings/*`	`settings.*`

顶部栏: Logo + 面包屑 + 通知铃铛（ `GET /api/notifications/unread-count` ）+ 用户下拉菜单

二、通用组件规格

2.1 Flutter 通用 Widget

StatusTag

```
StatusTag(status: String)
└── Container(
      padding: EdgeInsets.symmetric(horizontal: 8, vertical: 2),
      decoration: BoxDecoration(
        color: _statusColor(context, status).withOpacity(0.1),
        borderRadius: BorderRadius.circular(4),
      ),
    )
    └── Text(statusLabel(status),
          style: TextStyle(color: _statusColor(context, status), fontSize: 12))
```

色彩映射: 通过 `Theme.of(context).extension<CustomColors>()` 获取语义色，禁止硬编码 `Colors.xxx` 。

状态语义	Flutter token	示例状态

已租/已核销/已通过	`customColors.success`	`leased` `paid` `approved`
即将到期/预警/待审核	`customColors.warning`	`expiring_soon` `pending`
空置/逾期/已拒绝	`colorScheme.error`	`vacant` `overdue` `rejected`
非可租/已作废	`colorScheme.outline`	`non_leasable` `cancelled`
执行中/处理中/草稿	`colorScheme.primary`	`active` `in_progress` `draft`

PaginatedListView（分页列表）

替代 uni-app 的 `scroll-view + wd-loadmore` ：

```
PaginatedListView<T>(
  cubit: PaginatedCubit<T>,
  itemBuilder: (context, item) → Widget,
)
└── BlocBuilder<PaginatedCubit<T>, PaginatedState<T>>
    └── switch (state)
        case initial/loading (首次):
          → Center(child: CircularProgressIndicator())
        case loaded:
          → RefreshIndicator(
              onRefresh: cubit.refresh,
              child: ListView.builder(
                controller: _scrollController,  // 触底加载更多
                itemCount: state.items.length + (state.hasMore ? 1 : 0),
                itemBuilder: (ctx, i) =>
                  i < state.items.length
                    ? itemBuilder(ctx, state.items[i])
                    : _LoadMoreIndicator(),
              ),
            )
        case error:
          → ErrorRetryWidget(message: state.message, onRetry: cubit.fetch)
```

MetricCard

```
MetricCard(title: String, value: String, {subtitle, onTap})
└── Card(
      child: InkWell(
        onTap: onTap,
        child: Padding(
          child: Column(crossAxisAlignment: start,
            children: [
              Text(title, style: theme.textTheme.bodySmall),
              Text(value, style: theme.textTheme.headlineMedium),
              if (subtitle != null) Text(subtitle, style: bodySmall.copyWith(color: outline)),
            ],
          ),
        ),
      ),
    )
```

FilterChipBar（筛选标签栏）

替代 uni-app 的 `wd-drop-menu` / `wd-tag` 筛选行：

```
FilterChipBar(
  options: List<FilterOption>,
  selected: String?,
  onSelected: (String?) → void,
)
└── SingleChildScrollView(scrollDirection: Axis.horizontal)
    └── Row(
          children: options.map((opt) =>
            Padding(
              padding: EdgeInsets.only(right: 8),
              child: FilterChip(
                label: Text(opt.label),
                selected: selected == opt.value,
                onSelected: (_) => onSelected(opt.value),
              ),
            ),
          ),
        )
```

表单输入模式

Flutter 表单统一使用 `Form` + `TextFormField` + `GlobalKey<FormState>` ：

```
// 标准表单 Cubit 提交模式
Future<void> submit() async {
  if (!formKey.currentState!.validate()) return;
  emit(XxxState.loading());
  try {
    final result = await repository.create(form);
    emit(XxxState.loaded(result));
  } catch (e) {
    emit(XxxState.error(e is ApiException ? e.message : '操作失败，请重试'));
  }
}
```

BlocBuilder 状态渲染模板

所有页面 **必须** 使用 Dart 3 `switch` expression 或 `.when()` 渲染四态，禁止散落 `if (state is Xxx)` ：

```
BlocBuilder<XxxCubit, XxxState>(
  builder: (context, state) => switch (state) {
    XxxInitial() => const SizedBox.shrink(),
    XxxLoading() => const Center(child: CircularProgressIndicator()),
    XxxLoaded(:final data) => _buildContent(data),
    XxxError(:final message) => ErrorRetryWidget(
      message: message,
      onRetry: () => context.read<XxxCubit>().fetch(),
    ),
  },
```

```
)
```

2.2 Admin 通用组件

> 与 v1.8 完全一致：`ProposTable`（ElTable 封装 + 分页）、`StatusTag`（ElTag type 映射）、`MetricCard`（ElStatistic 封装）、`ElForm(inline)` 筛选栏模式。

三、认证模块页面

3.1 登录页

Admin: `LoginView.vue` — 路由 `/login`

Flutter: `LoginPage` — 路由 `/login`

Store/Cubit: Admin `useAuthStore` / Flutter `AuthCubit`

API: `POST /api/auth/login`

Admin 组件树：

```
LoginView
└── div.login-wrapper
    ├── div.brand-panel (品牌侧)
    └── ElCard
        └── ElForm(:model="form" :rules="rules")
            ├── ElFormItem("用户名") → ElInput(prefix-icon="User")
            ├── ElFormItem("密码") → ElInput(type="password" show-password)
            ├── ElAlert(v-if="error" type="error" :title="error")
            └── ElButton(type="primary" :loading="loading") "登录"
```

Flutter Widget 树：

```
LoginPage
└── BlocProvider(create: getIt<AuthCubit>())
    └── Scaffold
        └── SafeArea
            └── Padding(padding: 24)
                └── Column(mainAxisAlignment: center)
                    ├── Image.asset('assets/logo.png', height: 80)
                    ├── SizedBox(height: 32)
                    ├── Text("PropOS", style: headlineLarge)
                    ├── SizedBox(height: 48)
                    ├── Form(key: _formKey)
                    │   ├── TextFormField(
                    │   │     decoration: InputDecoration(labelText: "用户名", prefixIcon:
Icon(Icons.person)),
                    │   │     validator: requiredValidator,
                    │   │   )
                    │   ├── SizedBox(height: 16)
                    │   └── TextFormField(
```

```
                    │         decoration: InputDecoration(labelText: "密码", prefixIcon:
Icon(Icons.lock)),
                    │         obscureText: true,
                    │         validator: requiredValidator,
                    │       )
                    ├── SizedBox(height: 8)
                    ├── BlocBuilder<AuthCubit, AuthState>(
                    │     builder: (ctx, state) => switch (state) {
                    │       AuthError(:final message) =>
                    │         Container(padding: 8, child: Text(message, style:
TextStyle(color: colorScheme.error))),
                    │       _ => const SizedBox.shrink(),
                    │     },
                    │   )
                    ├── SizedBox(height: 24)
                    └── BlocConsumer<AuthCubit, AuthState>(
                          listener: (ctx, state) {
                            if (state is AuthLoaded) ctx.go('/dashboard');
                          },
                          builder: (ctx, state) => FilledButton(
                            onPressed: state is AuthLoading ? null : () => _submit(ctx),
                            child: state is AuthLoading
                              ? SizedBox(width: 20, height: 20, child:
CircularProgressIndicator(strokeWidth: 2))
                              : Text("登录"),
                          ),
                        )
```

3.2 修改密码页

Admin: `ChangePasswordView.vue` （用户下拉菜单触发 `ElDialog` ）

Flutter: `ChangePasswordPage` — 路由 `/change-password`

> 首次登录强制改密时， `AuthCubit` 检测 `must_change_password == true` 后 `context.go('/change-password')` ，路由守卫阻止跳转到其他页面。

***Flutter Widget* 树：**

```
ChangePasswordPage
└── Scaffold(appBar: AppBar(title: "修改密码"))
    └── Padding(padding: 24)
        └── Form(key: _formKey)
            ├── TextFormField(labelText: "旧密码", obscureText: true)
            ├── SizedBox(height: 16)
            ├── TextFormField(labelText: "新密码", obscureText: true,
            │     validator: passwordStrengthValidator)
            ├── SizedBox(height: 16)
            ├── TextFormField(labelText: "确认密码", obscureText: true,
            │     validator: confirmPasswordValidator)
            ├── SizedBox(height: 24)
            └── BlocConsumer<AuthCubit, AuthState>(
                  listener: ...,
```

```
                builder: (ctx, state) => FilledButton(
                  onPressed: () => ctx.read<AuthCubit>().changePassword(...),
                  child: Text("确认修改"),
                ),
              )
```

四、Dashboard 首页模块

4.1 首页

Admin: `DashboardView.vue` — 路由 `/dashboard`

Flutter: `DashboardPage` — 路由 `/dashboard`

Store/Cubit: Admin `useDashboardStore` / Flutter `DashboardCubit`

API: `GET /api/dashboard/overview`

Admin 组件树：

```
DashboardView
└── div
    ├── ElRow(:gutter="24")  — 核心指标卡片
    │   ├── MetricCard("总面积", "40,000 m²")
    │   ├── MetricCard("出租率", "92.3%")
    │   ├── MetricCard("NOI", "¥1,234,567", @click → noi-detail)
    │   └── MetricCard("WALE", "3.2 年", @click → wale-detail)
    │
    ├── ElRow(:gutter="24")  — 业态出租率
    │   ├── ElCard: 写字楼 ElProgress
    │   ├── ElCard: 商铺 ElProgress
    │   └── ElCard: 公寓 ElProgress
    │
    ├── ElCard(header="到期预警")
    │   └── ElTable(top 10 即将到期合同)
    │
    └── ElCard(header="逾期账单")
        └── ElTable(top 10 逾期账单)
```

Flutter Widget 树：

```
DashboardPage
└── BlocProvider(create: getIt<DashboardCubit>()..fetch())
    └── Scaffold(
          appBar: AppBar(
            title: Text("PropOS"),
            actions: [
              // 通知铃铛
              BlocBuilder<NotificationCubit, NotificationState>(
                builder: (ctx, state) => Badge(
```

```
                isLabelVisible: state.unreadCount > 0,
                label: Text('${state.unreadCount}'),
                child: IconButton(
                  icon: Icon(Icons.notifications_outlined),
                  onPressed: () => ctx.push('/notifications'),
                ),
              ),
            ),
          ],
        ),
        body: BlocBuilder<DashboardCubit, DashboardState>(
          builder: (ctx, state) => switch (state) {
            DashboardInitial() || DashboardLoading() =>
              const Center(child: CircularProgressIndicator()),
            DashboardLoaded(:final data) => _DashboardContent(data: data),
            DashboardError(:final message) => ErrorRetryWidget(
              message: message, onRetry: () => ctx.read<DashboardCubit>().fetch()),
          },
        ),
      )

_DashboardContent
└── RefreshIndicator(onRefresh: cubit.fetch)
    └── ListView(
          children: [
            // — 核心指标卡片 —
            Padding(padding: 16)
            └── Wrap(spacing: 12, runSpacing: 12,
                  children: [
                    MetricCard("总面积", "${data.totalArea} m²"),
                    MetricCard("出租率", "${data.occupancyRate}%"),
                    MetricCard("NOI", "¥${data.noi}", onTap: → /dashboard/noi-detail),
                    MetricCard("WALE", "${data.wale} 年", onTap: → /dashboard/wale-detail),
                  ],
                ),

            // — 业态出租率 —
            SizedBox(height: 16),
            Padding(padding: horizontal 16)
            └── Row(children: data.typeStats.map((t) =>
                  Expanded(child: _OccupancyMiniCard(type: t.type, rate: t.rate))
                )),

            // — 到期预警 —
            SizedBox(height: 16),
            _SectionHeader("到期预警"),
            ...data.expiringContracts.map((c) =>
              ListTile(
                title: Text(c.contractNumber),
                subtitle: Text("${c.tenantName} · 到期: ${formatDate(c.endDate)}"),
                trailing: StatusTag(status: c.status),
                onTap: () => ctx.push('/contracts/${c.id}'),
```

```
            ),
          ),

          // — 逾期账单 —
          _SectionHeader("逾期账单"),
          ...data.overdueInvoices.map((inv) =>
            ListTile(
              title: Text(inv.invoiceNumber),
              subtitle: Text("${inv.tenantName} · ¥${inv.amount}"),
              trailing: Text("逾期 ${inv.overdueDays} 天",
                style: TextStyle(color: colorScheme.error)),
              onTap: () => ctx.push('/finance/invoices'),
            ),
          ),
        ],
      )
```

4.2 NOI 明细页

Admin: `NoiDetailView.vue` — 路由 `/dashboard/noi-detail`

Flutter: `NoiDetailPage` — 路由 `/dashboard/noi-detail`

Store/Cubit: Admin `useNoiDetailStore` / Flutter `NoiDetailCubit`

API: `GET /api/noi/detail`

> NOI 明细页 Admin 端提供完整 ECharts 图表 + 明细表格；Flutter 端提供简化版指标卡片 + 列表。

***Admin* 组件树：**

```
NoiDetailView
└── div
    ├── ElRow(:gutter="24") — 指标卡片
    │   ├── MetricCard("PGI", "¥xxx")
    │   ├── MetricCard("空置损失", "¥xxx")
    │   ├── MetricCard("EGI", "¥xxx")
    │   ├── MetricCard("OpEx", "¥xxx")
    │   └── MetricCard("NOI", "¥xxx", subtitle: "Margin xx%")
    │
    ├── ElCard(header="NOI 月度趋势")
    │   └── ECharts(type: bar+line, NOI 月度趋势 + 预算对比线)
    │
    └── ElCard(header="NOI 明细分解")
        └── ElTable: 费项 | 金额 | 占比
```

***Flutter Widget* 树：**

```
NoiDetailPage
└── BlocProvider(create: getIt<NoiDetailCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "NOI 明细"))
        └── BlocBuilder<NoiDetailCubit, NoiDetailState>(
              builder: (ctx, state) => switch (state) {
```

```
                NoiDetailLoaded(:final data) =>
                  ListView(children: [
                    // 公式提示
                    Card(child: Padding(child: Text("NOI = EGI - OpEx = (PGI - 空置损失 + 其他收
入) - OpEx"))),
                    // 指标卡片
                    _MetricRow([
                      MetricCard("PGI", "¥${data.pgi}"),
                      MetricCard("空置损失", "¥${data.vacancyLoss}"),
                    ]),
                    _MetricRow([
                      MetricCard("EGI", "¥${data.egi}"),
                      MetricCard("OpEx", "¥${data.opex}"),
                    ]),
                    MetricCard("NOI", "¥${data.noi}", subtitle: "Margin ${data.noiMargin}%"),
                    // 分项列表
                    _SectionHeader("NOI 分解"),
                    ...data.breakdown.map((item) =>
                      ListTile(title: Text(item.category), trailing: Text("¥${item.amount}")),
                    ),
                  ]),
                ...
              },
            )
```

4.3 WALE 明细页

Admin: `WaleDetailView.vue` — 路由 `/dashboard/wale-detail`

Flutter: `WaleDetailPage` — 路由 `/dashboard/wale-detail`

Store/Cubit: Admin `useWaleDetailStore` / Flutter `WaleDetailCubit`

API: `GET /api/wale/detail`

Flutter Widget 树：

```
WaleDetailPage
└── BlocProvider(create: getIt<WaleDetailCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "WALE 明细"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                WaleDetailLoaded(:final data) =>
                  ListView(children: [
                    Card(child: Text("WALE = Σ(剩余租期ᵢ × 年化租金ᵢ) / Σ(年化租金ᵢ)")),
                    MetricCard("整体 WALE", "${data.overallWale} 年"),
                    // 分业态
                    ...data.byType.map((t) =>
                      ListTile(title: Text(t.propertyType), trailing: Text("${t.wale} 年")),
                    ),
                    _SectionHeader("合同明细"),
                    ...data.contracts.map((c) =>
                      ListTile(
```

```
                    title: Text(c.contractNumber),
                    subtitle: Text("${c.tenantName} · 剩余 ${c.remainingYears} 年"),
                    trailing: Text("¥${c.annualizedRent}/年"),
                  ),
                ),
              ]),
            ...
          },
        )
```

4.4 KPI 考核看板

Admin: `KpiView.vue` — 路由 `/finance/kpi`

Flutter: `KpiDashboardPage` — 路由 `/finance/kpi`

Store/Cubit: Admin `useKpiDashboardStore` / Flutter `KpiDashboardCubit`

API: `GET /api/kpi/scores` + `GET /api/kpi/rankings`

***Admin* 组件树：**

```
KpiView
└── div
    ├── ElRow(:gutter="24") — 核心指标
    │   ├── MetricCard("综合得分", "87.5", subtitle: "排名 2/8")
    │   ├── MetricCard("出租率得分", "92 × 30% = 27.6")
    │   └── MetricCard("NOI 达成率", "85 × 25% = 21.25")
    │
    ├── ElCard(header="指标明细")
    │   └── ElTable: 指标 | 权重 | 实际值 | 得分 | 满分标准 | 及格标准
    │
    ├── ElCard(header="团队排名")
    │   └── ElTable: 排名 | 姓名 | 部门 | 总分
    │
    └── ElButton(icon="Download") "导出 KPI 报告"
```

***Flutter Widget* 树：**

```
KpiDashboardPage
└── BlocProvider(create: getIt<KpiDashboardCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "KPI 考核"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                KpiDashboardLoaded(:final data) =>
                  ListView(children: [
                    // 综合得分卡片
                    Card(
                      child: Column(children: [
                        Text("综合得分", style: bodySmall),
                        Text("${data.totalScore}", style: displayMedium),
                        Text("排名 ${data.rank}/${data.totalMembers}"),
```

```
                ]),
              ),
              // 指标明细
              _SectionHeader("指标明细"),
              ...data.metrics.map((m) =>
                ListTile(
                  title: Text(m.name),
                  subtitle: Text("权重 ${m.weight}% · 实际 ${m.actualValue}"),
                  trailing: Text("${m.score}", style: titleMedium),
                ),
              ),
              // 排名
              _SectionHeader("团队排名"),
              ...data.rankings.map((r) =>
                ListTile(
                  leading: CircleAvatar(child: Text("${r.rank}")),
                  title: Text(r.name),
                  subtitle: Text(r.department),
                  trailing: Text("${r.totalScore}"),
                ),
              ),
            ]),
          ...
        },
      )
```

五、资产与空间模块页面

5.1 资产总览页

Admin: `AssetsView.vue` — 路由 `/assets`

Flutter: `AssetsPage` — 路由 `/assets`

Store/Cubit: Admin `useAssetOverviewStore` / Flutter `AssetOverviewCubit`

API: `GET /api/buildings` + `GET /api/assets/statistics`

***Admin* 组件树：**

```
AssetsView
└── div
    ├── ElRow(:gutter="24") — 业态统计
    │   ├── MetricCard("写字楼", "xxx m² · 出租率 xx%")
    │   ├── MetricCard("商铺", "xxx m² · 出租率 xx%")
    │   └── MetricCard("公寓", "xxx m² · 出租率 xx%")
    │
    └── ElRow(:gutter="24")
        └── ElCard(v-for="building in buildings")
            ├── ElText(tag="h3") building.name
            ├── ElDescriptions: 地址 | 总层数 | 总面积
```

```
            └── ElButton(type="primary" link) "查看详情"
```

Flutter Widget 树：

```
AssetsPage
└── BlocProvider(create: getIt<AssetOverviewCubit>()..fetch())
    └── Scaffold(
          appBar: AppBar(title: "资产管理"),
          body: BlocBuilder<AssetOverviewCubit, AssetOverviewState>(
            builder: (ctx, state) => switch (state) {
              AssetOverviewLoaded(:final data) =>
                RefreshIndicator(
                  onRefresh: ctx.read<AssetOverviewCubit>().fetch,
                  child: ListView(children: [
                    // 业态统计卡片
                    Padding(padding: 16,
                      child: Row(children: data.typeStats.map((t) =>
                        Expanded(child: MetricCard(
                          t.typeName, "${t.area} m²",
                          subtitle: "出租率 ${t.occupancyRate}%",
                        )),
                      ).toList()),
                    ),
                    // 楼栋列表
                    ...data.buildings.map((b) =>
                      Card(
                        margin: EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                        child: ListTile(
                          title: Text(b.name, style: titleMedium),
                          subtitle: Text("${b.totalFloors} 层 · ${b.totalArea} m²"),
                          trailing: Icon(Icons.chevron_right),
                          onTap: () => ctx.push('/assets/buildings/${b.id}'),
                        ),
                      ),
                    ),
                  ]),
                ),
              ...
            },
          ),
        )
```

5.2 楼栋详情页

Admin: `BuildingDetailView.vue` — 路由 `/assets/buildings/:id`

Flutter: `BuildingDetailPage` — 路由 `/assets/buildings/:id`

Store/Cubit: Admin `useBuildingDetailStore` / Flutter `BuildingDetailCubit`

API: `GET /api/buildings/:id` + `GET /api/floors?building_id=`

Admin 组件树：

```
BuildingDetailView
└── div
    ├── ElDescriptions(border :column="2" title="楼栋信息")
    │   ├── 名称 | 地址 | 总层数 | 总面积 | 出租率
    │
    └── ElCard(header="楼层列表")
        └── ElTable(:data="floors" @row-click="toFloorPlan")
            ├── ElTableColumn(label="楼层") "{{ floor.floor_number }}F"
            ├── ElTableColumn(label="总单元数")
            ├── ElTableColumn(label="已租")
            ├── ElTableColumn(label="空置")
            └── ElTableColumn(label="出租率") → ElProgress
```

Flutter Widget 树：

```
BuildingDetailPage
└── BlocProvider(create: getIt<BuildingDetailCubit>()..fetch(buildingId))
    └── Scaffold(appBar: AppBar(title: "楼栋详情"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                BuildingDetailLoaded(:final building, :final floors) =>
                  ListView(children: [
                    // 楼栋信息卡片
                    Card(margin: 16,
                      child: Padding(padding: 16,
                        child: Column(crossAxisAlignment: start, children: [
                          Text(building.name, style: titleLarge),
                          SizedBox(height: 8),
                          _InfoRow("地址", building.address),
                          _InfoRow("总层数", "${building.totalFloors} 层"),
                          _InfoRow("总面积", "${building.totalArea} m²"),
                          _InfoRow("出租率", "${building.occupancyRate}%"),
                        ]),
                      ),
                    ),
                    // 楼层列表
                    _SectionHeader("楼层列表"),
                    ...floors.map((f) =>
                      ListTile(
                        leading: CircleAvatar(child: Text("${f.floorNumber}F")),
                        title: Text("${f.unitCount} 个单元"),
                        subtitle: LinearProgressIndicator(
                          value: f.occupancyRate / 100,
                          backgroundColor: colorScheme.surfaceVariant,
                        ),
                        trailing: Text("${f.occupancyRate}%"),
                        onTap: () => ctx.push(
                          '/assets/buildings/${building.id}/floors/${f.id}'),
                      ),
```

```
                ),
              ]),
            ...
          },
        )
```

5.3 楼层热区图页

Admin: `FloorPlanView.vue` — 路由 `/assets/buildings/:bid/floors/:fid`

Flutter: `FloorPlanPage` — 路由 `/assets/buildings/:bid/floors/:fid`

Store/Cubit: Admin `useFloorMapStore` / Flutter `FloorMapCubit`

API: `GET /api/floors/:id/plan` (SVG) + `GET /api/floors/:id/units`

Admin 组件树：

```
FloorPlanView
└── div.floor-plan-layout (display: flex)
    ├── div.unit-list (width: 300px)
    │   └── ElTable(:data="units" @row-click="highlightUnit"): 单元号 | 状态 | 面积
    │
    └── div.svg-container (flex: 1)
        ├── div(v-html="svgContent")  // SVG 热区，各 polygon 按单元状态着色
        └── div.legend: 已租(绿) | 即将到期(黄) | 空置(红) | 非可租(灰)
```

Flutter Widget 树：

> Flutter 端通过 `WebViewWidget`（`webview_flutter`）加载 SVG + 交互脚本。HarmonyOS Next 使用系统 WebView。

```
FloorPlanPage
└── BlocProvider(create: getIt<FloorMapCubit>()..fetch(floorId))
    └── Scaffold(appBar: AppBar(title: "楼层热区图"))
        └── BlocBuilder<FloorMapCubit, FloorMapState>(
              builder: (ctx, state) => switch (state) {
                FloorMapLoaded(:final svgUrl, :final units) =>
                  Column(children: [
                    // — SVG 渲染区 —
                    Expanded(
                      child: WebViewWidget(
                        controller: _webViewController
                          ..loadRequest(Uri.parse(svgUrl)),
                        // JS channel 接收单元点击事件
                      ),
                    ),
                    // — 图例 —
                    _LegendBar(),  // 已租(绿) | 即将到期(黄) | 空置(红) | 非可租(灰)
                    // — 底部单元详情弹窗 —
                    // 通过 showModalBottomSheet 展示
                  ]),
                ...
```

```
            },
          )
```

底部弹窗（点击热区单元触发）：

```
showModalBottomSheet(
  context: context,
  builder: (_) => SizedBox(
    height: MediaQuery.of(context).size.height * 0.4,
    child: Column(children: [
      ListTile(title: Text("单元"), trailing: Text(unit.unitNumber)),
      ListTile(title: Text("面积"), trailing: Text("${unit.area} m²")),
      ListTile(title: Text("状态"), trailing: StatusTag(status: unit.status)),
      ListTile(title: Text("租户"), trailing: Text(unit.tenantName ?? "—")),
      Divider(),
      Row(mainAxisAlignment: spaceEvenly, children: [
        FilledButton(onPressed: () => ctx.push('/assets/units/${unit.id}'), child: Text("查看详
情")),
        OutlinedButton(onPressed: () => ctx.push('/contracts/${unit.contractId}'), child:
Text("查看合同")),
      ]),
    ]),
  ),
);
```

5.4 房源详情页

Admin: `UnitDetailView.vue` — 路由 `/assets/units/:id`

Flutter: `UnitDetailPage` — 路由 `/assets/units/:id`

Store/Cubit: Admin `useUnitDetailStore` / Flutter `UnitDetailCubit`

API: `GET /api/units/:id` + `GET /api/renovations?unit_id=`

Admin 组件树：

```
UnitDetailView
└── div
    ├── ElSpace: StatusTag(status) + ElTag(property_type) + ElButton("编辑")
    ├── ElDescriptions(border :column="2" title="基本信息")
    │   ├── 单元编号 | 建筑面积 | 套内面积 | 朝向 | 层高 | 装修状态 | 市场租金 | 前序单元
    ├── ElDescriptions(title="业态信息")  // 根据 property_type 动态字段
    ├── ElCard(header="当前租赁")  // v-if status=leased
    │   └── ElDescriptions: 租户 | 合同编号 | 月租金 | 到期日
    └── ElCard(header="改造记录")
        └── ElTable: 改造类型 | 日期 | 造价
```

Flutter Widget 树：

```
UnitDetailPage
└── BlocProvider(create: getIt<UnitDetailCubit>()..fetch(unitId))
```

```
    └── Scaffold(appBar: AppBar(title: "房源详情"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                UnitDetailLoaded(:final unit, :final renovations) =>
                  ListView(children: [
                    // 状态栏
                    Padding(padding: 16,
                      child: Wrap(spacing: 8, children: [
                        StatusTag(status: unit.currentStatus),
                        Chip(label: Text(unit.propertyTypeLabel)),
                      ]),
                    ),
                    // 基本信息
                    _SectionHeader("基本信息"),
                    ListTile(title: Text("单元编号"), trailing: Text(unit.unitNumber)),
                    ListTile(title: Text("建筑面积"), trailing: Text("${unit.gfa} m²")),
                    ListTile(title: Text("套内面积"), trailing: Text("${unit.nia} m²")),
                    ListTile(title: Text("市场租金"),
                      trailing: Text("¥${unit.marketRentReference}/m²/月")),
                    // 当前租赁信息 (仅 leased 显示)
                    if (unit.currentStatus == 'leased') ...[
                      _SectionHeader("当前租赁"),
                      ListTile(title: Text("租户"), trailing: Text(unit.tenantName!)),
                      ListTile(title: Text("月租金"), trailing: Text("¥${unit.monthlyRent}")),
                      ListTile(
                        title: Text("到期日"), trailing: Text(formatDate(unit.endDate!)),
                        onTap: () => ctx.push('/contracts/${unit.contractId}'),
                      ),
                    ],
                    // 改造记录
                    _SectionHeader("改造记录"),
                    ...renovations.map((r) =>
                      ListTile(
                        title: Text(r.type),
                        subtitle: Text(formatDate(r.date)),
                        trailing: Text("¥${r.cost}"),
                      ),
                    ),
                  ]),
                ...
              },
            )
```

5.5 Excel 批量导入页

> 批量导入仅 Admin 端提供，Flutter 端不支持。

***Admin* 组件树：**

```
UnitImportView
└── div
```

```
    └── ElSteps(:active="currentStep" align-center)
        ├── ElStep("选择文件"): ElSelect(数据类型) + ElUpload + ElLink("下载模板")
        ├── ElStep("预校验"): ElButton("试导入 dry_run") + 校验结果 ElTable
        ├── ElStep("确认导入"): 确认提示 + ElProgress
        └── ElStep("治理与回滚"): 批次管理 ElTable + 批量修正 ElUpload
```

六、租务与合同模块页面

6.1 合同列表页

Admin: `ContractsView.vue` — 路由 `/contracts`

Flutter: `ContractsPage` — 路由 `/contracts` （TabBar 第 3 Tab）

Store/Cubit: Admin `useContractListStore` / Flutter `ContractListCubit`

API: `GET /api/contracts`

Admin 组件树：

```
ContractsView
└── div
    ├── ElForm(inline)
    │   ├── ElSelect "状态: 全部/报价中/执行中/即将到期/已终止..."
    │   ├── ElSelect "业态" + ElSelect "楼栋"
    │   ├── ElInput "搜索: 合同编号/租户名称"
    │   └── ElButton(type="primary" icon="Plus") "新建合同"
    │
    └── ProposTable
        ├── 合同编号(sortable) | 租户 | 业态→ElTag | 单元 | 月租金
        ├── 状态→StatusTag | 到期日
        └── 操作: ElDropdown(续签/终止/查看押金)
```

Flutter Widget 树：

```
ContractsPage
└── BlocProvider(create: getIt<ContractListCubit>()..fetch())
    └── Scaffold(
          appBar: AppBar(title: "合同管理"),
          body: Column(children: [
            // — 筛选栏 —
            Padding(padding: horizontal 16,
              child: Column(children: [
                // 搜索框
                TextField(
                  decoration: InputDecoration(
                    hintText: "搜索合同/租户",
                    prefixIcon: Icon(Icons.search),
                  ),
                  onChanged: (v) => cubit.updateKeyword(v),
```

```
                ),
                SizedBox(height: 8),
                // 状态筛选
                FilterChipBar(
                  options: contractStatusOptions,
                  selected: state.filters.status,
                  onSelected: (v) => cubit.filterByStatus(v),
                ),
              ]),
            ),
            // — 合同列表 —
            Expanded(
              child: BlocBuilder<ContractListCubit, ContractListState>(
                builder: (ctx, state) => switch (state) {
                  ContractListLoaded(:final contracts, :final hasMore) =>
                    PaginatedListView(
                      items: contracts,
                      hasMore: hasMore,
                      onLoadMore: () => cubit.loadMore(),
                      onRefresh: () => cubit.refresh(),
                      itemBuilder: (ctx, contract) =>
                        Card(
                          margin: EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                          child: ListTile(
                            title: Row(children: [
                              Text(contract.contractNumber, style: titleSmall),
                              Spacer(),
                              StatusTag(status: contract.status),
                            ]),
                            subtitle: Column(crossAxisAlignment: start, children: [
                              Text(contract.tenantName),
                              Text("¥${contract.monthlyRent}/月 · 到期 $
{formatDate(contract.endDate)}"),
                            ]),
                            onTap: () => ctx.push('/contracts/${contract.id}'),
                          ),
                        ),
                    ),
                  ...
                },
              ),
            ),
          ]),
        )
```

6.2 合同新建/编辑页

> 合同新建/编辑为复杂表单，仅 **Admin** 端提供；Flutter 端只提供查看合同详情功能。

***Admin* 组件树：**

```
ContractFormView
└── ElForm(label-width="120px")
```

```
    ├── Section "租户信息": ElSelect(filterable remote) + "新建租户" 链接
    ├── Section "合同基本信息": 合同编号/付款周期/起租日/到期日/免租天数/装修期/含税/税率
    ├── Section "关联单元(M:N)": ElTable(可编辑: 计费面积+单价) + 合计行
    ├── Section "押金": ElInputNumber
    ├── Section "商铺分成" (v-if retail): 保底月租金 + 分成比例
    └── Section "附件上传": ElUpload(multiple .pdf)
```

6.3 合同详情页

Admin: `ContractDetailView.vue` — 路由 `/contracts/:contractId`

Flutter: `ContractDetailPage` — 路由 `/contracts/:id`

Store/Cubit: Admin `useContractDetailStore` / Flutter `ContractDetailCubit`

API: `GET /api/contracts/:id` + `GET /api/contracts/:id/escalation-phases` + `GET /api/contracts/:id/attachments`

Admin 组件树:

```
ContractDetailView
└── ElTabs(v-model="activeTab")
    ├── Tab "基本信息": StatusTag + ElDescriptions(合同信息) + 关联单元 ElTable + 续签链 ElTimeline
    ├── Tab "递增规则": ElTimeline(各阶段) + "编辑递增规则" 按钮
    ├── Tab "押金": ElDescriptions(总额/余额/状态) + 交易流水 ElTable + 操作(冻结/冲抵/退还/转移)
    ├── Tab "子租赁" (仅二房东主合同): ElTable(单元/终端租客/月租金/审核状态)
    └── Tab "附件": ElTable(文件名/大小/上传时间/操作) + ElUpload
```

Flutter Widget 树:

```
ContractDetailPage
└── BlocProvider(create: getIt<ContractDetailCubit>()..fetch(contractId))
    └── Scaffold(appBar: AppBar(title: "合同详情"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                ContractDetailLoaded(:final contract, :final phases, :final deposits) =>
                  DefaultTabController(
                    length: 3,
                    child: Column(children: [
                      // 状态栏
                      Padding(padding: 16,
                        child: Row(children: [
                          StatusTag(status: contract.status),
                          SizedBox(width: 8),
                          Chip(label: Text(contract.propertyTypeLabel)),
                        ]),
                      ),
                      // Tab 切换
                      TabBar(tabs: [
                        Tab(text: "基本信息"),
                        Tab(text: "递增规则"),
                        Tab(text: "押金"),
```

```
                      ]),
                      // Tab 内容
                      Expanded(child: TabBarView(children: [
                        // — Tab 1: 基本信息 —
                        ListView(children: [
                          ListTile(title: Text("合同编号"), trailing:
Text(contract.contractNumber)),
                          ListTile(title: Text("租户"), trailing: Text(contract.tenantName)),
                          ListTile(title: Text("起租日"), trailing:
Text(formatDate(contract.startDate))),
                          ListTile(title: Text("到期日"), trailing:
Text(formatDate(contract.endDate))),
                          ListTile(title: Text("月租金"), trailing: Text("¥$
{contract.monthlyRent}")),
                          ListTile(title: Text("付款周期"), trailing:
Text(contract.paymentCycle)),
                          // 关联单元
                          _SectionHeader("关联单元"),
                          ...contract.units.map((u) =>
                            ListTile(
                              title: Text(u.unitNumber),
                              subtitle: Text("${u.area}m² · ¥${u.unitPrice}/m²/月"),
                              onTap: () => ctx.push('/assets/units/${u.id}'),
                            ),
                          ),
                        ]),
                        // — Tab 2: 递增规则 —
                        ListView(children: [
                          ...phases.asMap().entries.map((e) =>
                            _PhaseTimelineItem(
                              index: e.key + 1,
                              phase: e.value,
                            ),
                          ),
                        ]),
                        // — Tab 3: 押金 —
                        ListView(children: [
                          ListTile(title: Text("押金总额"), trailing: Text("¥$
{deposits.total}")),
                          ListTile(title: Text("当前余额"), trailing: Text("¥$
{deposits.balance}")),
                        ]),
                      ])),
                    ]),
                  ),
                ...
              },
            )
```

6.4 合同终止页

> 仅 Admin 端提供。

***Admin* 组件树：**

```
ContractTerminateView
└── ElForm(label-width="120px")
    ├── 合同概要（只读）ElDescriptions
    ├── 终止信息：终止类型/终止日期/终止原因
    ├── 违约金 & 押金处理：违约金 + 当前押金余额 + 扣除金额 + 预计退还
    ├── 影响预览：ElAlert(warning) 列出终止影响
    └── ElButton(type="danger") "确认终止" → ElMessageBox.confirm
```

6.5 递增规则配置页

> 仅 Admin 端提供。

***Admin* 组件树：**

```
EscalationConfigView
└── div
    ├── "从模板套用" 按钮
    ├── 递增阶段列表(v-for phase)：ElCard(日期范围 + 递增类型 + 参数)
    │   递增类型：固定比例/固定金额/阶梯式/CPI挂钩/每N年/免租后基准
    ├── "添加阶段" 按钮
    └── 租金预测预览：ElTable(年份/月租金/年化租金/涨幅)
```

6.5.1 租金预测页（Should: S-01）

> 仅 Admin 端提供。路由 `/contracts/:contractId/rent-forecast` 。

6.6 租客列表页

> 仅 Admin 端提供。

***Admin* 组件树：**

```
TenantListView
└── div
    ├── ElForm(inline)：搜索(名称/证件号后4位) + 信用评级 + 类型 + "新建租客"
    └── ProposTable：名称 | 类型 | 证件号(脱敏) | 信用评级→StatusTag | 在租合同数 | 联系人
```

6.7 租客详情页

> 仅 Admin 端提供。

***Admin* 组件树：**

```
TenantDetailView
└── div
```

```
    ├── ElDescriptions(border): 名称/类型/证件号(脱敏+解锁按钮)/联系人/联系电话(脱敏+解锁)/信用评级
    ├── ElCard("缴费信用"): 当前评级/评级日期/逾期次数 + 评级历史趋势(Should)
    ├── ElCard("租赁历史"): ElTable(合同编号/单元/起止日期/状态)
    └── ElCard("报修工单"): ElTable(最近工单)
```

6.8 押金管理

> 押金管理在合同详情页 Tab 3 中集成。独立操作通过弹窗完成（冻结/冲抵/退还/转移），仅 **Admin** 端。

6.9 合同续签页

> 仅 **Admin** 端提供。

***Admin* 组件树：**

```
ContractRenewView
└── ElForm(label-width="120px")
    ├── 原合同信息（只读）
    ├── 续签参数: 新起始日/新到期日/新月租金/递增规则(延用 or 重配)
    ├── 押金处理: 自动转入/退还重收/补差额
    └── "提交续签" → ElMessageBox.confirm
```

6.10 租客新增/编辑页

> 仅 **Admin** 端提供。

***Admin* 组件树：**

```
TenantFormView
└── ElForm(label-width="120px")
    ├── 基本信息: 名称/类型/证件号(⚠️加密存储)
    ├── 联系人: 姓名/电话(⚠️加密存储)/邮箱
    ├── 开票信息(可选): 抬头/税号/开户行/银行账号
    └── 提交按钮
```

七、财务模块页面

7.1 财务概览页

Admin: `FinanceView.vue` — 路由 `/finance`

Flutter: `FinancePage` — 路由 `/finance` （TabBar 第 5 Tab）

Store/Cubit: Admin `useFinanceOverviewStore` / Flutter `FinanceOverviewCubit`

API: `GET /api/noi/summary` + `GET /api/invoices?status=overdue`

***Admin* 组件树：**

```
FinanceView
```

```
└── div
    ├── ElRow: MetricCard(本月应收/本月实收/收款率/NOI)
    ├── ElRow: ActionCard(账单管理/费用支出/水电抄表/营业额申报)
    ├── ElCard("逾期账单"): ElTable(top 10)
    └── ElCard("本月收款进度"): ElProgress
```

Flutter Widget 树（角色差异化视图）：

> 财务页根据登录用户角色渲染差异化视图。通过 `AuthCubit` 获取当前角色，在 `FinanceOverviewCubit` 中决定请求数据范围和 UI 呈现。

```
FinancePage
└── MultiBlocProvider(
      providers: [
        BlocProvider(create: getIt<FinanceOverviewCubit>()..fetch()),
        BlocProvider(create: getIt<AuthCubit>()),
      ],
    )
    └── Scaffold(
          appBar: AppBar(title: "财务"),
          body: BlocBuilder<AuthCubit, AuthState>(
            builder: (ctx, authState) =>
              BlocBuilder<FinanceOverviewCubit, FinanceOverviewState>(
                builder: (ctx, state) => switch (state) {
                  FinanceOverviewLoaded(:final data) =>
                    _buildRoleView(ctx, authState.role, data),
                  ...
                },
              ),
          ),
        )
```

管理层视图（`super_admin` / `operations_manager`）：

```
_ManagerFinanceView
└── RefreshIndicator
    └── ListView(children: [
          // Header — 深蓝渐变容器
          _GradientHeader(
            gradient: [Color(0xFF0F2645), Color(0xFF1A3A5C)],
            title: "财务概览·经营看板",
          ),
          // NOI + WALE 摘要卡
          _NoiSummaryCard(data.noi, onTap: → /dashboard/noi-detail),
          _WaleSummaryCard(data.wale, onTap: → /dashboard/wale-detail),
          _RevenueSnapshotCard(data.revenue),
          // 核心待办
          _SectionHeader("核心待办"),
          Row(children: [
            Expanded(child: _FeaturedCard("KPI 申诉", badge: data.kpiAppealCount,
              onTap: → /finance/kpi)),
```

```
            SizedBox(width: 12),
            Expanded(child: _FeaturedCard("待审批", badge: data.pendingApprovalCount,
              onTap: → /approvals)),
          ]),
          // 二级入口
          _SecondaryIconRow([
            _IconEntry(Icons.receipt, "费用", → /finance/invoices),
            _IconEntry(Icons.water_drop, "水电", → /finance/dunning),
            _IconEntry(Icons.account_balance_wallet, "押金", → /finance/invoices),
            _IconEntry(Icons.store, "营业额", → /finance/invoices),
            _IconEntry(Icons.warning, "催收", → /finance/dunning),
          ]),
          // 逾期账单 Top 5
          _OverdueSection(data.overdueInvoices),
        ])
```

财务专员视图（ `finance_staff` ）：

```
_FinanceStaffView
└── ListView(children: [
      _GradientHeader(gradient: [Color(0xFF064E3B), Color(0xFF065F46)], title: "今日待处理"),
      _SectionHeader("今日任务"),
      Row(children: [
        Expanded(child: _FeaturedCard("账单核销", badge: data.pendingPayments,
          onTap: → /finance/invoices)),
        Expanded(child: _FeaturedCard("水电审核", badge: data.pendingMeterReviews,
          onTap: → /finance/invoices)),
      ]),
      _SecondaryIconRow([费用/押金/营业额/NOI预算/催收]),
      _OverdueSection(data.overdueInvoices),
    ])
```

租务专员视图（ `leasing_specialist` ）：

```
_LeasingStaffView
└── ListView(children: [
      _GradientHeader(gradient: [Color(0xFF1A3A5C), Color(0xFF2A5298)], title: "租务财务"),
      _SectionHeader("我的事项"),
      Row(children: [
        Expanded(child: _FeaturedCard("押金管理", onTap: → /finance/invoices)),
        Expanded(child: _FeaturedCard("营业额申报", onTap: → /finance/invoices)),
      ]),
      _SecondaryIconRow([水电/账单/KPI]),
      _CompactCollectionWidget(data.collectionProgress),
    ])
```

维修技工视图（ `maintenance_staff` ）：

```
_MaintenanceStaffView
└── ListView(children: [
```

```
    _GradientHeader(gradient: [Color(0xFF78350F), Color(0xFF92400E)], title: "水电录入"),
    _SectionHeader("待录入"),
    _FeaturedCard("水电录入", badge: data.pendingMeterEntries, fullWidth: true,
      onTap: → /finance/invoices),
    _SecondaryIconRow([账单查看/KPI]),
  ])
```

楼管巡检 / 只读观察: 类似结构，差异化 Header 渐变色和功能入口。

共用子 **Widget** 清单:

Widget	说明
`_GradientHeader`	渐变背景 + 标题文字
`_SectionHeader`	彩色竖条 + 区块标题
`_FeaturedCard`	图标 + Badge + 2 行摘要 + onTap
`_SecondaryIconRow`	小图标横排，含可选 Badge
`_NoiSummaryCard`	深色 NOI 摘要卡
`_WaleSummaryCard`	深色 WALE 摘要卡
`_RevenueSnapshotCard`	收入快报卡
`_OverdueSection`	逾期账单 Top 5 列表
`_CompactCollectionWidget`	进度条 + 已收/应收（租务专员视图）

7.2 账单列表页

Admin: `InvoicesView.vue` — 路由 `/finance/invoices`

Flutter: `InvoiceListPage` — 路由 `/finance/invoices`

Store/Cubit: Admin `useInvoiceListStore` / Flutter `InvoiceListCubit`

API: `GET /api/invoices`

Admin 组件树:

```
InvoicesView
└── div
    ├── ElForm(inline): 状态/费项/楼栋/业态/账期范围/租户搜索 + "导出Excel" + "手工触发生成"
    └── ProposTable: 账单号 | 租户 | 费项 | 含税金额 | 不含税金额 | 状态→StatusTag | 到期日 | 操作(核销/作废)
```

Flutter Widget 树:

```
InvoiceListPage
└── BlocProvider(create: getIt<InvoiceListCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "账单列表"))
        └── Column(children: [
```

```
            // 状态筛选标签栏
            Padding(padding: horizontal 16,
              child: FilterChipBar(
                options: [全部, 已出账, 逾期, 已核销],
                selected: state.filters.status,
                onSelected: (v) => cubit.filterByStatus(v),
              ),
            ),
            // 账单列表
            Expanded(
              child: BlocBuilder<InvoiceListCubit, InvoiceListState>(
                builder: (ctx, state) => switch (state) {
                  InvoiceListLoaded(:final invoices, :final hasMore) =>
                    PaginatedListView(
                      items: invoices,
                      hasMore: hasMore,
                      onLoadMore: cubit.loadMore,
                      onRefresh: cubit.refresh,
                      itemBuilder: (ctx, inv) =>
                        ListTile(
                          leading: StatusTag(status: inv.status),
                          title: Text(inv.invoiceNumber),
                          subtitle: Text("${inv.tenantName} · ${inv.billingPeriod}"),
                          trailing: Text("¥${inv.totalAmount}"),
                        ),
                    ),
                  ...
                },
              ),
            ),
          ])
```

7.3 ~ 7.11 财务子页面

> 以下财务子页面仅 **Admin** 端提供完整功能（账单详情、收款录入、水电抄表、营业额申报、费用管理、押金台账、NOI 预算），Flutter 移动端通过财务概览页的入口卡片导航至对应 Admin 功能。

7.3 账单详情页 — Admin: `InvoiceDetailView.vue`

7.4 收款录入页 — Admin: `PaymentFormView.vue`

7.5 水电抄表录入页 — Admin: `MeterReadingFormView.vue` （支持独立表/公区分摊两种模式）

7.6 营业额申报管理页 — Admin: `TurnoverReportListView.vue` + `TurnoverReportDetailView.vue`

7.7 费用列表页 — Admin: `ExpenseListView.vue`

7.8 费用录入页 — Admin: `ExpenseFormView.vue`

7.9 水电抄表列表页 — Admin: `MeterReadingListView.vue`

7.10 押金台账页 — Admin: `DepositListView.vue`

7.11 NOI 预算管理页 — Admin: `NoiBudgetView.vue`

> Admin 组件树与 v1.8 完全一致，不再重复。

八、工单模块页面

> Phase 1 支持三种工单类型：`repair`（报修）、`complaint`（投诉）、`inspection`（退租验房）。

8.1 工单列表页

Admin: `WorkordersView.vue` — 路由 `/workorders`

Flutter: `WorkordersPage` — 路由 `/workorders`（TabBar 第 4 Tab）

Store/Cubit: Admin `useWorkOrderListStore` / Flutter `WorkOrderListCubit`

API: `GET /api/workorders`

Admin 组件树：

```
WorkordersView
└── div
    ├── ElForm(inline)
    │   ├── ElRadioGroup "工单类型：全部/报修/投诉/退租验房"
    │   ├── ElRadioGroup "状态：全部/已提交/处理中/待验收/已完成/挂起"
    │   ├── ElInput "搜索" + ElButton "新建工单"
    │
    └── ProposTable：工单编号 | 类型→ElTag | 描述 | 位置 | 优先级→ElTag | 状态→StatusTag | 处理人 | 提报时间
```

Flutter Widget 树：

```
WorkordersPage
└── BlocProvider(create: getIt<WorkOrderListCubit>()..fetch())
    └── Scaffold(
          appBar: AppBar(title: "工单管理"),
          floatingActionButton: FloatingActionButton(
            onPressed: () => _showNewOrderOptions(ctx),
            child: Icon(Icons.add),
          ),
          body: Column(children: [
            // 工单类型切换
            Padding(padding: horizontal 16,
              child: SegmentedButton<String>(
                segments: [
                  ButtonSegment(value: '', label: Text('全部')),
                  ButtonSegment(value: 'repair', label: Text('报修')),
                  ButtonSegment(value: 'complaint', label: Text('投诉')),
                  ButtonSegment(value: 'inspection', label: Text('验房')),
                ],
                selected: {state.filters.type},
                onSelectionChanged: (v) => cubit.filterByType(v.first),
              ),
            ),
            SizedBox(height: 8),
```

```
            // 状态筛选标签栏
            FilterChipBar(
              options: workOrderStatusOptions,
              selected: state.filters.status,
              onSelected: (v) => cubit.filterByStatus(v),
            ),
            // 工单列表
            Expanded(
              child: BlocBuilder<WorkOrderListCubit, WorkOrderListState>(
                builder: (ctx, state) => switch (state) {
                  WorkOrderListLoaded(:final orders, :final hasMore) =>
                    PaginatedListView(
                      items: orders,
                      hasMore: hasMore,
                      onLoadMore: cubit.loadMore,
                      onRefresh: cubit.refresh,
                      itemBuilder: (ctx, order) =>
                        Card(
                          margin: EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                          child: ListTile(
                            title: Row(children: [
                              Expanded(child: Text(order.title, maxLines: 1, overflow:
ellipsis)),
                              StatusTag(status: order.status),
                            ]),
                            subtitle: Column(crossAxisAlignment: start, children: [
                              Text("${order.building} ${order.floor} ${order.unit}"),
                              Row(children: [
                                Text("提报: ${order.submitter}"),
                                Spacer(),
                                Text(timeAgo(order.submittedAt),
                                  style: TextStyle(color: colorScheme.outline)),
                              ]),
                            ]),
                            leading: _priorityIcon(order.priority),
                            onTap: () => ctx.push('/workorders/${order.id}'),
                          ),
                        ),
                    ),
                  ...
                },
              ),
            ),
          ]),
        )
```

FAB 点击逻辑（Flutter）：

```
void _showNewOrderOptions(BuildContext ctx) {
  showModalBottomSheet(
    context: ctx,
    builder: (_) => SafeArea(
      child: Column(mainAxisSize: MainAxisSize.min, children: [
```

```
        ListTile(
          leading: Icon(Icons.qr_code_scanner),
          title: Text("扫码报修"),
          onTap: () {
            Navigator.pop(ctx);
            _scanAndNavigate(ctx);
          },
        ),
        ListTile(
          leading: Icon(Icons.build),
          title: Text("手动报修"),
          onTap: () {
            Navigator.pop(ctx);
            ctx.push('/workorders/new?type=repair');
          },
        ),
        ListTile(
          leading: Icon(Icons.report_problem),
          title: Text("提交投诉"),
          onTap: () {
            Navigator.pop(ctx);
            ctx.push('/workorders/new?type=complaint');
          },
        ),
        ListTile(
          leading: Icon(Icons.home_repair_service),
          title: Text("退租验房"),
          onTap: () {
            Navigator.pop(ctx);
            ctx.push('/workorders/new?type=inspection');
          },
        ),
      ]),
    ),
  );
}
```

8.2 工单提报页

Admin: `WorkorderFormView.vue` — 路由 `/workorders/new`

Flutter: `WorkorderFormPage` — 路由 `/workorders/new`

Store/Cubit: Admin `useWorkOrderFormStore` / Flutter `WorkOrderFormCubit`

API: `POST /api/workorders`

***Admin* 组件树：**

```
WorkorderFormView
└── ElForm(label-width="120px")
    ├── 工单类型：ElRadioGroup(报修/投诉/退租验房)
    ├── 位置选择（级联）：楼栋→楼层→单元
    ├── 关联合同(仅 inspection)：ElSelect(filterable)
```

```
    ├── 问题描述 + 问题类型 + 紧急程度
    ├── 现场照片: ElUpload(picture-card, limit 5)
    └── "提交工单"
```

Flutter Widget 树:

```
WorkorderFormPage
└── BlocProvider(create: getIt<WorkOrderFormCubit>()..init(type: queryType))
    └── Scaffold(appBar: AppBar(title: "提交工单"))
        └── BlocConsumer<WorkOrderFormCubit, WorkOrderFormState>(
              listener: (ctx, state) {
                if (state is WorkOrderFormSubmitted) {
                  ScaffoldMessenger.of(ctx).showSnackBar(SnackBar(content: Text("工单已提交")));
                  ctx.pop();
                }
              },
              builder: (ctx, state) =>
                SingleChildScrollView(
                  padding: EdgeInsets.all(16),
                  child: Form(key: _formKey, child: Column(
                    crossAxisAlignment: start,
                    children: [
                      // 工单类型
                      Text("工单类型", style: titleSmall),
                      SizedBox(height: 8),
                      SegmentedButton<String>(
                        segments: [
                          ButtonSegment(value: 'repair', label: Text('报修')),
                          ButtonSegment(value: 'complaint', label: Text('投诉')),
                          ButtonSegment(value: 'inspection', label: Text('验房')),
                        ],
                        selected: {state.form.workOrderType},
                        onSelectionChanged: (v) => cubit.setType(v.first),
                      ),
                      SizedBox(height: 16),
                      // 位置选择（级联）
                      DropdownButtonFormField<String>(
                        decoration: InputDecoration(labelText: "楼栋"),
                        items: state.buildings.map((b) => DropdownMenuItem(value: b.id, child:
Text(b.name))).toList(),
                        onChanged: (v) => cubit.setBuilding(v!),
                        validator: requiredValidator,
                      ),
                      SizedBox(height: 16),
                      DropdownButtonFormField<String>(
                        decoration: InputDecoration(labelText: "楼层"),
                        items: state.floors.map((f) => DropdownMenuItem(value: f.id, child:
Text("${f.floorNumber}F"))).toList(),
                        onChanged: (v) => cubit.setFloor(v!),
                      ),
                      SizedBox(height: 16),
```

```
                      DropdownButtonFormField<String>(
                        decoration: InputDecoration(labelText: "单元"),
                        items: state.units.map((u) => DropdownMenuItem(value: u.id, child:
Text(u.unitNumber))).toList(),
                        onChanged: (v) => cubit.setUnit(v!),
                      ),
                      // 关联合同（仅 inspection）
                      if (state.form.workOrderType == 'inspection') ...[
                        SizedBox(height: 16),
                        DropdownButtonFormField<String>(
                          decoration: InputDecoration(labelText: "关联合同"),
                          items: state.contracts.map((c) => DropdownMenuItem(value: c.id,
child: Text(c.contractNumber))).toList(),
                          onChanged: (v) => cubit.setContract(v!),
                        ),
                      ],
                      SizedBox(height: 16),
                      // 问题描述
                      TextFormField(
                        decoration: InputDecoration(labelText: "问题描述", alignLabelWithHint:
true),
                        maxLines: 5, maxLength: 500,
                        validator: requiredValidator,
                        onChanged: (v) => cubit.setDescription(v),
                      ),
                      SizedBox(height: 16),
                      // 问题类型
                      DropdownButtonFormField<String>(
                        decoration: InputDecoration(labelText: "问题类型"),
                        items: state.issueTypeOptions.map((t) => DropdownMenuItem(value:
t.value, child: Text(t.label))).toList(),
                        onChanged: (v) => cubit.setCategory(v!),
                      ),
                      SizedBox(height: 16),
                      // 紧急程度
                      Text("紧急程度", style: titleSmall),
                      SizedBox(height: 8),
                      SegmentedButton<String>(
                        segments: [
                          ButtonSegment(value: 'normal', label: Text('一般')),
                          ButtonSegment(value: 'urgent', label: Text('紧急')),
                          ButtonSegment(value: 'critical', label: Text('非常紧急')),
                        ],
                        selected: {state.form.priority},
                        onSelectionChanged: (v) => cubit.setPriority(v.first),
                      ),
                      SizedBox(height: 16),
                      // 现场照片
                      Text("现场照片", style: titleSmall),
                      SizedBox(height: 8),
                      _PhotoPicker(
```

```
                        photos: state.form.photos,
                        limit: 5,
                        onAdd: cubit.addPhoto,
                        onRemove: cubit.removePhoto,
                      ),
                      SizedBox(height: 24),
                      // 提交按钮
                      SizedBox(
                        width: double.infinity,
                        child: FilledButton(
                          onPressed: state is WorkOrderFormSubmitting ? null : () =>
_submit(ctx),
                          child: state is WorkOrderFormSubmitting
                            ? CircularProgressIndicator(strokeWidth: 2)
                            : Text("提交工单"),
                        ),
                      ),
                    ],
                  )),
                ),
              )
```

8.3 工单详情页

Admin: `WorkorderDetailView.vue` — 路由 `/workorders/:orderId`

Flutter: `WorkorderDetailPage` — 路由 `/workorders/:id`

Store/Cubit: Admin `useWorkOrderDetailStore` / Flutter `WorkOrderDetailCubit`

API: `GET /api/workorders/:id`

Admin 组件树：

```
WorkorderDetailView
└── div
    ├── ElSpace: StatusTag + ElTag(priority)
    ├── ElDescriptions(border): 工单编号/位置/问题类型/提报人/提报时间/处理人/SLA状态
    ├── ElCard("问题描述")
    ├── ElCard("现场照片"): ElImage(preview)
    ├── ElDescriptions("维修成本") — completed 状态
    ├── ElCard("操作记录"): ElTimeline
    └── 操作按钮(根据状态): 审核派单/开始处理/提交完工/验收通过/返工/重开
```

Flutter Widget 树：

```
WorkorderDetailPage
└── BlocProvider(create: getIt<WorkOrderDetailCubit>()..fetch(orderId))
    └── Scaffold(appBar: AppBar(title: "工单详情"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                WorkOrderDetailLoaded(:final order, :final logs, :final photos) =>
                  Column(children: [
```

```
                    Expanded(child: ListView(children: [
                      // 状态/优先级
                      Padding(padding: 16,
                        child: Row(children: [
                          StatusTag(status: order.status),
                          SizedBox(width: 8),
                          Chip(label: Text(order.priorityLabel)),
                        ]),
                      ),
                      // 基本信息
                      ListTile(title: Text("工单编号"), trailing: Text(order.orderNumber)),
                      ListTile(title: Text("位置"), trailing: Text(order.locationText)),
                      ListTile(title: Text("问题类型"), trailing: Text(order.category)),
                      ListTile(title: Text("提报人"), trailing: Text(order.submitter)),
                      ListTile(title: Text("处理人"), trailing: Text(order.assignee ?? "—")),
                      // 问题描述
                      Padding(padding: 16, child: Card(
                        child: Padding(padding: 16, child: Text(order.description)),
                      )),
                      // 照片
                      if (photos.isNotEmpty) ...[
                        _SectionHeader("现场照片"),
                        SizedBox(
                          height: 100,
                          child: ListView.builder(
                            scrollDirection: Axis.horizontal,
                            itemCount: photos.length,
                            itemBuilder: (ctx, i) => GestureDetector(
                              onTap: () => _previewImage(ctx, photos, i),
                              child: Padding(
                                padding: EdgeInsets.only(left: 16),
                                child: ClipRRect(
                                  borderRadius: BorderRadius.circular(8),
                                  child: Image.network(photos[i], width: 100, height: 100, fit:
BoxFit.cover),
                                ),
                              ),
                            ),
                          ),
                        ),
                      ],
                      // 操作时间线
                      _SectionHeader("操作记录"),
                      ...logs.map((log) =>
                        ListTile(
                          leading: Icon(Icons.circle, size: 12, color: colorScheme.primary),
                          title: Text(log.description),
                          subtitle: Text(formatDateTime(log.time)),
                        ),
                      ),
                    ])),
                    // 底部操作按钮
```

```
                    _WorkOrderActionBar(status: order.status, orderId: order.id),
                  ]),
                ...
              },
            )
```

底部操作栏（根据工单状态动态显示）：

```
_WorkOrderActionBar(status, orderId)
└── SafeArea(
      child: Padding(padding: 16,
        child: switch (status) {
          'submitted' => FilledButton(onPressed: → 审核派单弹窗, child: Text("审核派单")),
          'approved' => FilledButton(onPressed: → start, child: Text("开始处理")),
          'in_progress' => FilledButton(onPressed: → complete, child: Text("提交完工")),
          'pending_inspection' => Row(children: [
            Expanded(child: FilledButton(onPressed: → pass, child: Text("验收通过"))),
            SizedBox(width: 12),
            Expanded(child: OutlinedButton(onPressed: → rework, child: Text("返工"))),
          ]),
          _ => const SizedBox.shrink(),
        },
      ),
    )
```

8.4 扫码报修

> Flutter 端通过 `mobile_scanner` 包实现 QR 扫码。HarmonyOS Next 使用系统相机 API。

```
// 扫码流程
Future<void> _scanAndNavigate(BuildContext ctx) async {
  final result = await ctx.push<String>('/scanner');
  if (result == null) return;
  try {
    final unit = await getIt<UnitRepository>().getByQrCode(result);
    ctx.push('/workorders/new?unit_id=${unit.id}&building_id=${unit.buildingId}&floor_id=$
{unit.floorId}');
  } catch (e) {
    ScaffoldMessenger.of(ctx).showSnackBar(
      SnackBar(content: Text('无法识别二维码，请重试或手动输入')),
    );
  }
}
```

九、二房东门户模块页面

> 二房东模块分两个交付面：**内部管理视角**（Admin + Flutter 辅助查看）与**外部门户视角**（独立 Web）。

9.1 二房东管理列表（内部管理视角）

Admin: `SubleasesView.vue` — 路由 `/subleases`

Flutter: `SubleasesPage` — 路由 `/subleases` （内部员工辅助查看）

Store/Cubit: Admin `useSubleaseListStore` / Flutter `SubleaseListCubit`

API: `GET /api/subleases`

***Admin* 组件树：**

```
SubleasesView
└── div
    ├── "新建子租赁" + "批量导入" 按钮
    ├── ElForm(inline): 审核状态/二房东/搜索
    └── ProposTable: 单元 | 二房东 | 终端租客 | 月租金 | 入住状态 | 审核状态→StatusTag | 操作
```

***Flutter Widget* 树：**

```
SubleasesPage
└── BlocProvider(create: getIt<SubleaseListCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "二房东管理"))
        └── Column(children: [
              FilterChipBar(options: [全部, 待审核, 已通过, 已退回], ...),
              Expanded(
                child: BlocBuilder(
                  builder: (ctx, state) => switch (state) {
                    SubleaseListLoaded(:final items, :final hasMore) =>
                      PaginatedListView(
                        items: items, hasMore: hasMore,
                        onLoadMore: cubit.loadMore, onRefresh: cubit.refresh,
                        itemBuilder: (ctx, item) =>
                          Card(margin: EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                            child: ListTile(
                              title: Text("${item.unitNumber} · ${item.endTenantName}"),
                              subtitle: Text("二房东: ${item.subLandlordName} · ¥${item.monthlyRent}/月"),
                              trailing: StatusTag(status: item.reviewStatus),
                              onTap: () => ctx.push('/subleases/${item.id}'),
                            ),
                          ),
                      ),
                    ...
                  },
                ),
              ),
            ])
```

9.2 子租赁详情页（内部管理视角）

Admin: `SubleaseDetailView.vue` — 路由 `/subleases/:id`

Flutter: `SubleaseDetailPage` — 路由 `/subleases/:id`

Flutter Widget 树：

```
SubleaseDetailPage
└── BlocProvider(create: getIt<SubleaseDetailCubit>()..fetch(subleaseId))
    └── Scaffold(appBar: AppBar(title: "子租赁详情"))
        └── BlocBuilder(
              builder: (ctx, state) => switch (state) {
                SubleaseDetailLoaded(:final item, :final logs) =>
                  ListView(children: [
                    _SectionHeader("基本信息"),
                    ListTile(title: Text("主合同"), trailing: Text(item.masterContractNumber)),
                    ListTile(title: Text("单元"), trailing: Text(item.unitNumber)),
                    ListTile(title: Text("终端租客"), trailing: Text(item.endTenantName)),
                    ListTile(title: Text("审核状态"), trailing: StatusTag(status:
item.reviewStatus)),
                    _SectionHeader("租赁信息"),
                    ListTile(title: Text("起租日"), trailing: Text(formatDate(item.startDate))),
                    ListTile(title: Text("到期日"), trailing: Text(formatDate(item.endDate))),
                    ListTile(title: Text("月租金"), trailing: Text("¥${item.monthlyRent}")),
                    _SectionHeader("变更记录"),
                    ...logs.map((log) => ListTile(
                      leading: Icon(Icons.circle, size: 12),
                      title: Text(log.description),
                      subtitle: Text(formatDateTime(log.time)),
                    )),
                  ]),
                ...
              },
            )
```

9.3 ~ 9.7 二房东子页面

> 子租赁录入/编辑页 (9.3)、二房东登录页 (9.4)、单元填报列表页 (9.5)、子租赁填报页 (9.6)、批量导入页 (9.7) — 均为 **Admin / 外部门户 Web** 专属页面，Flutter 端不提供。

Admin 组件树与 v1.8 完全一致，不再重复。

十、通知与审批模块页面

10.1 通知中心

Admin: `NotificationCenterView.vue` — 路由 `/notifications`

Flutter: `NotificationsPage` — 路由 `/notifications`

Store/Cubit: Admin `useNotificationStore` / Flutter `NotificationCubit`

API: `GET /api/notifications` + `PATCH /api/notifications/:id/read` + `PATCH /api/notifications/read-all`

权限: `notifications.read` （所有角色）

Admin 组件树：

```
NotificationCenterView
└── div
    ├── 顶部工具栏：类型筛选 + 级别筛选 + 状态筛选 + "全部已读"
    └── ElTable(@row-click): Badge(未读点) | 级别→ElTag | 类型→ElTag | 标题 | 时间 | 操作
        ElPagination
```

Flutter Widget 树：

```
NotificationsPage
└── BlocProvider(create: getIt<NotificationCubit>()..fetch())
    └── Scaffold(
          appBar: AppBar(
            title: Text("通知中心"),
            actions: [
              TextButton(
                onPressed: () => cubit.markAllRead(),
                child: Text("全部已读"),
              ),
            ],
          ),
          body: Column(children: [
            // Tab 筛选
            Padding(padding: horizontal 16,
              child: SegmentedButton<String>(
                segments: [
                  ButtonSegment(value: 'all', label: Text('全部')),
                  ButtonSegment(value: 'unread', label: Text('未读')),
                  ButtonSegment(value: 'critical', label: Text('重要')),
                ],
                selected: {state.activeTab},
                onSelectionChanged: (v) => cubit.switchTab(v.first),
              ),
            ),
            // 通知列表
            Expanded(
              child: BlocBuilder<NotificationCubit, NotificationState>(
                builder: (ctx, state) => switch (state) {
                  NotificationLoaded(:final items, :final hasMore) =>
                    items.isEmpty
                      ? Center(child: Column(mainAxisSize: min, children: [
                          Icon(Icons.notifications_off, size: 64, color: colorScheme.outline),
                          SizedBox(height: 16),
                          Text("暂无通知"),
                        ]))
                      : PaginatedListView(
                          items: items, hasMore: hasMore,
                          onLoadMore: cubit.loadMore, onRefresh: cubit.refresh,
                          itemBuilder: (ctx, item) =>
                            ListTile(
                              leading: Badge(
```

```
                              isLabelVisible: !item.isRead,
                              child: Icon(_notificationIcon(item.type)),
                            ),
                            title: Text(item.title,
                              style: TextStyle(
                                fontWeight: item.isRead ? FontWeight.normal :
FontWeight.bold,
                              )),
                            subtitle: Text(timeAgo(item.createdAt)),
                            trailing: _severityChip(item.severity),
                            onTap: () {
                              cubit.markRead(item.id);
                              _goToResource(ctx, item);
                            },
                          ),
                        ),
                    ...
                  },
                ),
              ),
            ]),
          )
```

10.2 审批队列

Admin: `ApprovalQueueView.vue` — 路由 `/approvals`

Flutter: `ApprovalsPage` — 路由 `/approvals`

Store/Cubit: Admin `useApprovalStore` / Flutter `ApprovalCubit`

API: `GET /api/approvals` + `PATCH /api/approvals/:id`

权限: `approvals.manage` （仅 SA / OM）

Admin 组件树:

```
ApprovalQueueView
└── div
    ├── ElForm(inline): 审批类型 + 状态 + 日期范围
    ├── ElRow: MetricCard(待审批/本周已审/本周已拒)
    └── ElTable: 类型→ElTag | 申请人 | 摘要 | 申请时间 | 状态→StatusTag | 操作(通过/拒绝/查看详情)
```

Flutter Widget 树:

```
ApprovalsPage
└── BlocProvider(create: getIt<ApprovalCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "审批队列"))
        └── Column(children: [
              // Tab 筛选
              Padding(padding: horizontal 16,
                child: SegmentedButton<String>(
                  segments: [
                    ButtonSegment(value: 'pending', label: Text('待审批')),
```

```
                ButtonSegment(value: 'approved', label: Text('已通过')),
                ButtonSegment(value: 'rejected', label: Text('已退回')),
              ],
              selected: {state.activeTab},
              onSelectionChanged: (v) => cubit.switchTab(v.first),
            ),
          ),
          // 审批列表
          Expanded(
            child: BlocBuilder<ApprovalCubit, ApprovalState>(
              builder: (ctx, state) => switch (state) {
                ApprovalLoaded(:final items, :final hasMore) =>
                  PaginatedListView(
                    items: items, hasMore: hasMore,
                    onLoadMore: cubit.loadMore, onRefresh: cubit.refresh,
                    itemBuilder: (ctx, item) =>
                      Card(
                        margin: EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                        child: Padding(padding: 16,
                          child: Column(crossAxisAlignment: start, children: [
                            Row(children: [
                              Chip(label: Text(approvalTypeLabel(item.approvalType))),
                              Spacer(),
                              StatusTag(status: item.status),
                            ]),
                            SizedBox(height: 8),
                            Text(item.summary, style: bodyMedium),
                            SizedBox(height: 4),
                            Text("${item.requesterName} · $
{formatDateTime(item.createdAt)}",
                              style: bodySmall.copyWith(color: outline)),
                            if (item.status == 'pending') ...[
                              SizedBox(height: 12),
                              Row(children: [
                                OutlinedButton(
                                  onPressed: () => _goToResource(ctx, item),
                                  child: Text("查看详情"),
                                ),
                                Spacer(),
                                FilledButton.tonal(
                                  onPressed: () => cubit.approve(item.id),
                                  child: Text("通过"),
                                ),
                                SizedBox(width: 8),
                                FilledButton(
                                  style: FilledButton.styleFrom(
                                    backgroundColor: colorScheme.error),
                                  onPressed: () => _showRejectSheet(ctx, item),
                                  child: Text("拒绝"),
                                ),
                              ]),
                            ],
                          ]),
```

```
                            ),
                          ),
                        ),
                      ...
                    },
                  ),
                ),
              ])
```

拒绝理由弹窗（Flutter）：

```
void _showRejectSheet(BuildContext ctx, ApprovalItem item) {
  showModalBottomSheet(
    context: ctx,
    isScrollControlled: true,
    builder: (_) => Padding(
      padding: EdgeInsets.only(
        bottom: MediaQuery.of(ctx).viewInsets.bottom,
        left: 16, right: 16, top: 16,
      ),
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        TextField(
          controller: _remarkController,
          decoration: InputDecoration(labelText: "请输入拒绝理由"),
          maxLines: 3,
        ),
        SizedBox(height: 16),
        SizedBox(width: double.infinity,
          child: FilledButton(
            onPressed: () {
              ctx.read<ApprovalCubit>().reject(item.id, _remarkController.text);
              Navigator.pop(ctx);
            },
            child: Text("确认提交"),
          ),
        ),
        SizedBox(height: 16),
      ]),
    ),
  );
}
```

十一、催收管理页面

11.1 催收记录列表

Admin: `DunningListView.vue` — 路由 `/finance/dunning`

Flutter: `DunningListPage` — 路由 `/finance/dunning`

Store/Cubit: Admin `useDunningStore` / Flutter `DunningCubit`

API: `GET /api/dunning-logs`

权限: `finance.read`

> 双端分层: Flutter 负责催收记录查看、待跟进查看；Admin 保留新建催收、编辑、复杂筛选和日志治理。

Admin 组件树:

```
DunningListView
└── div
    ├── ElForm(inline): 租客名称 + 催收方式 + 日期范围 + "新建催收"
    └── ProposTable: 催收日期 | 租客 | 关联账单 | 催收方式→ElTag | 催收金额 | 备注 | 操作人 | 下次跟进
```

Flutter Widget 树:

```
DunningListPage
└── BlocProvider(create: getIt<DunningCubit>()..fetch())
    └── Scaffold(appBar: AppBar(title: "催收记录"))
        └── Column(children: [
              // 顶部摘要卡
              Padding(padding: 16,
                child: Row(children: [
                  Expanded(child: MetricCard("待跟进", "${state.pendingCount}")),
                  SizedBox(width: 12),
                  Expanded(child: MetricCard("本周催收", "${state.weekCount}")),
                  SizedBox(width: 12),
                  Expanded(child: MetricCard("逾期金额", "¥${state.overdueAmount}")),
                ]),
              ),
              // Tab 筛选
              FilterChipBar(options: [全部, 待跟进, 已完成], ...),
              // 催收列表
              Expanded(
                child: BlocBuilder<DunningCubit, DunningState>(
                  builder: (ctx, state) => switch (state) {
                    DunningLoaded(:final items, :final hasMore) =>
                      PaginatedListView(
                        items: items, hasMore: hasMore,
                        onLoadMore: cubit.loadMore, onRefresh: cubit.refresh,
                        itemBuilder: (ctx, item) =>
                          Card(
                            margin: EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                            child: Padding(padding: 16,
                              child: Column(crossAxisAlignment: start, children: [
                                Row(children: [
                                  Text(item.tenantName, style: titleSmall),
                                  Spacer(),
                                  Chip(label: Text(dunningMethodLabel(item.method))),
                                ]),
                                SizedBox(height: 8),
                                _InfoRow("关联账单", item.invoiceNo),
                                _InfoRow("催收日期", formatDate(item.dunningDate)),
                                _InfoRow("下次跟进", item.nextFollowUp != null
```

```
                                ? formatDate(item.nextFollowUp!) : "—"),
                              SizedBox(height: 8),
                              Row(children: [
                                OutlinedButton(
                                  onPressed: () => ctx.push('/finance/invoices'),
                                  child: Text("查看账单"),
                                ),
                                SizedBox(width: 8),
                                TextButton(
                                  onPressed: () => _showRemark(ctx, item.remark),
                                  child: Text("查看备注"),
                                ),
                              ]),
                            ]),
                          ),
                        ),
                      ),
                    ...
                  },
                ),
              ),
            ])
```

十二、系统设置模块页面

> 系统设置仅 **Admin** 端提供，Flutter 端不包含设置页面。

12.1 ~ 12.9 设置子页面

所有设置页面仅 Admin 端，组件树与 v1.8 一致：

页面	Admin 路由	说明
用户管理	`/settings/users`	用户 CRUD + 角色分配
用户新建/编辑	`/settings/users/new`	表单：基本信息+角色+初始密码
组织架构管理	`/settings/org`	左树右详情：部门树 + 管辖范围 + 成员
KPI 方案管理	`/settings/kpi/schemes`	列表 + 步骤式表单(基本信息→指标配置→绑定对象)
KPI 申诉	`/settings/kpi/appeal`	员工提交申诉 + 管理层审核
递增模板管理	`/settings/escalation/templates`	模板列表 + CRUD
预警中心	`/settings/alerts`	预警记录 Tab + 失败任务 Tab + 补发弹窗
审计日志	`/settings/audit-logs`	操作类型/操作人/日期范围筛选 + 日志表

十三、响应式断点与布局策略

13.1 双端布局策略

Admin（PC Web）：

断点	宽度范围	侧边栏	布局列数
小屏	< 768px	折叠至 64px	1 列
中屏	768~1199px	折叠至 64px	2 列
大屏	≥ 1200px	展开 240px	2~4 列

Flutter（移动端）：

平台	布局	导航
iOS	全屏单列	NavigationBar + go_router 页面栈
Android	全屏单列	NavigationBar + go_router 页面栈
HarmonyOS Next	全屏单列	NavigationBar + go_router 页面栈

> Flutter Web 不在 Phase 1 范围内。

13.2 Admin 组件响应策略

组件	小屏 (< 768px)	大屏 (≥ 1200px)
`MetricCard` 行	2列	4列
`ElTable`	横向滚动	全列展示
表单	单列	双列
Tab	可横向滚动	全部可见
图表	宽高自适应	固定高度
楼层图	全屏查看	左侧列表+右侧图

13.3 平台能力降级策略

功能	iOS/Android	HarmonyOS Next	Admin PC
QR 扫码报修	✅ `mobile_scanner`	✅ 系统相机 API	❌ → 手动填报
推送通知	✅ FCM/APNs	✅ Push Kit	❌ → 轮询
相机拍照	✅ `image_picker`	✅	❌ → 文件选择
文件选取	✅	✅	✅
Excel 批量导入	❌ → Admin 操作	❌ → Admin 操作	✅
SVG 热区图	✅ WebViewWidget	✅ 系统 WebView	✅ v-html

十四、状态色语义映射速查

14.0 双端色彩体系

Admin（Element Plus 内置 type）：

type	色系	语义
`success`	绿色	已租/已核销/已通过/已完成
`warning`	黄/橙色	即将到期/预警/待审核
`danger`	红色	空置/逾期/错误/已拒绝
`info`	灰色	非可租/已作废/已停用
`primary`	蓝色	执行中/处理中/草稿

Flutter（Material 3 + ThemeExtension）：

token	色系	语义
`customColors.success`	绿色	已租/已核销/已通过/已完成
`customColors.warning`	黄/橙色	即将到期/预警/待审核
`colorScheme.error`	红色	空置/逾期/错误/已拒绝
`colorScheme.outline`	灰色	非可租/已作废/已停用
`colorScheme.primary`	蓝色	执行中/处理中/草稿

> Flutter 颜色 必须 通过 `Theme.of(context).colorScheme.*` 或 `Theme.of(context).extension<CustomColors>()` 获取，禁止 `Colors.green` / `Color(0xFF...)` 硬编码。

14.1 通用状态色

状态语义	Admin ElTag type	Flutter token	适用场景
已租/已核销/已通过/已完成	`success`	`customColors.success`	单元已租、账单已核销、审核通过
即将到期/预警/待审核	`warning`	`customColors.warning`	合同即将到期、逾期预警
空置/逾期/错误/已拒绝	`danger`	`colorScheme.error`	空置、逾期、审核退回
非可租/已作废/已停用	`info`	`colorScheme.outline`	非可租单元、作废账单
执行中/处理中/草稿	`primary`	`colorScheme.primary`	执行中、处理中

14.2 合同状态色映射

状态	Admin type	Flutter token	标签文案
`quoting`	`primary`	`colorScheme.primary`	报价中
`pending_sign`	`warning`	`customColors.warning`	待签约
`active`	`success`	`customColors.success`	执行中
`expiring_soon`	`warning`	`customColors.warning`	即将到期
`expired`	`info`	`colorScheme.outline`	已到期
`renewed`	`success`	`customColors.success`	已续签
`terminated`	`danger`	`colorScheme.error`	已终止

14.3 账单状态色映射

状态	Admin type	Flutter token	标签文案
`draft`	`primary`	`colorScheme.primary`	草稿
`issued`	`warning`	`customColors.warning`	已出账
`paid`	`success`	`customColors.success`	已核销
`overdue`	`danger`	`colorScheme.error`	逾期
`cancelled`	`info`	`colorScheme.outline`	已作废
`exempt`	`info`	`colorScheme.outline`	免租免单

14.4 工单状态色映射

状态	Admin type	Flutter token	标签文案
`submitted`	`primary`	`colorScheme.primary`	已提交
`approved`	`warning`	`customColors.warning`	已派单
`in_progress`	`primary`	`colorScheme.primary`	处理中
`pending_inspection`	`warning`	`customColors.warning`	待验收
`completed`	`success`	`customColors.success`	已完成
`rejected`	`danger`	`colorScheme.error`	已拒绝
`on_hold`	`info`	`colorScheme.outline`	挂起

14.5 信用评级色映射

评级	Admin type	Flutter token	标签文案
A	`success`	`customColors.success`	A 优质
B	`warning`	`customColors.warning`	B 一般
C	`danger`	`colorScheme.error`	C 风险
D	`danger`	`colorScheme.error`	D 严重违约

附录 A：页面清单与模块映射

A.1 Flutter 页面清单（21 个页面）

页面	go_router path	模块	TabBar
登录	`/login`	认证	—
首页	`/dashboard`	概览	✅ Tab 1
NOI 移动分析	`/dashboard/noi-detail`	概览	—
WALE 移动分析	`/dashboard/wale-detail`	概览	—
资产总览	`/assets`	资产	✅ Tab 2
楼栋详情	`/assets/buildings/:id`	资产	—
楼层热区图	`/assets/buildings/:bid/floors/:fid`	资产	—
房源详情	`/assets/units/:id`	资产	—
合同管理	`/contracts`	租务	✅ Tab 3
合同详情	`/contracts/:id`	租务	—
财务总览	`/finance`	财务	✅ Tab 5
账单列表	`/finance/invoices`	财务	—
KPI 考核	`/finance/kpi`	KPI	—
催收记录	`/finance/dunning`	财务	—
工单管理	`/workorders`	工单	✅ Tab 4
工单详情	`/workorders/:id`	工单	—
新建工单	`/workorders/new`	工单	—
二房东管理（内部）	`/subleases`	二房东	—
二房东详情（内部）	`/subleases/:id`	二房东	—
通知中心	`/notifications`	通知	—
审批队列	`/approvals`	审批	—

A.2 Admin 视图清单（49 视图 + 5 门户视图）

视图	路由	模块	优先级
`LoginView`	`/login`	认证	Must
`DashboardView`	`/dashboard`	概览	Must
`NoiDetailView`	`/dashboard/noi-detail`	概览	Must
`WaleDetailView`	`/dashboard/wale-detail`	概览	Must
`AssetsView`	`/assets`	资产	Must
`BuildingDetailView`	`/assets/buildings/:id`	资产	Must
`FloorPlanView`	`/assets/buildings/:bid/floors/:fid`	资产	Must
`UnitDetailView`	`/assets/units/:id`	资产	Must
`UnitImportView`	`/assets/import`	资产	Must
`ContractsView`	`/contracts`	租务	Must
`ContractFormView`	`/contracts/new`	租务	Must
`ContractDetailView`	`/contracts/:id`	租务	Must
`ContractTerminateView`	`/contracts/:id/terminate`	租务	Must
`ContractRenewView`	`/contracts/:id/renew`	租务	Must
`EscalationConfigView`	`/contracts/:id/escalation`	租务	Must
`RentForecastView`	`/contracts/:id/rent-forecast`	租务	Should
`TenantListView`	`/tenants`	租务	Must
`TenantDetailView`	`/tenants/:id`	租务	Must
`TenantFormView`	`/tenants/new`	租务	Must
`FinanceView`	`/finance`	财务	Must
`InvoicesView`	`/finance/invoices`	财务	Must
`InvoiceDetailView`	`/finance/invoices/:id`	财务	Must
`PaymentFormView`	`/finance/invoices/:id/pay`	财务	Must
`ExpenseListView`	`/finance/expenses`	财务	Must
`ExpenseFormView`	`/finance/expenses/new`	财务	Must
`MeterReadingListView`	`/finance/meter-readings`	财务	Must
`MeterReadingFormView`	`/finance/meter-readings/new`	财务	Must
`TurnoverReportListView`	`/finance/turnover-reports`	财务	Must
`TurnoverReportDetailView`	`/finance/turnover-reports/:id`	财务	Must
`KpiView`	`/finance/kpi`	KPI	Must
`KpiSchemeDetailView`	`/finance/kpi/scheme/:id`	KPI	Must

DepositListView	/finance/deposits	财务	Must
NoiBudgetView	/finance/noi-budget	财务	Must
DunningListView	/finance/dunning	财务	Must
WorkordersView	/workorders	工单	Must
WorkorderFormView	/workorders/new	工单	Must
WorkorderDetailView	/workorders/:id	工单	Must
SubleasesView	/subleases	二房东	Must
SubleaseDetailView	/subleases/:id	二房东	Must
SubleaseFormView	/subleases/new	二房东	Must
SubleaseImportView	/subleases/import	二房东	Must
NotificationCenterView	/notifications	通知	Must
ApprovalQueueView	/approvals	审批	Must
UserManagementView	/settings/users	设置	Must
UserFormView	/settings/users/new	设置	Must
OrganizationManageView	/settings/org	设置	Must
KpiSchemeListView	/settings/kpi/schemes	KPI	Must
KpiSchemeFormView	/settings/kpi/schemes/new	KPI	Must
KpiAppealView	/settings/kpi/appeal	KPI	Must
EscalationTemplateListView	/settings/escalation/templates	设置	Must
AlertCenterView	/settings/alerts	设置	Must
AuditLogView	/settings/audit-logs	设置	Must

二房东外部门户视图（**5** 个）：

视图	路由	优先级
PortalLoginView	/portal/login	Must
PortalChangePasswordView	/portal/change-password	Must
SubLandlordPortalListView	/portal/subleases	Must
SubleaseFillingView	/portal/subleases/:id/edit	Must
SubleaseImportView	/portal/subleases/import	Must

> 总计: Flutter **21** 个页面 + Admin **52** 个视图（含 3 个 v1.8 新增）+ 外部门户 **5** 个视图。

附录 B：BLoC/Cubit 清单（Flutter）+ Pinia Store 清单（Admin）

B.1 Flutter BLoC/Cubit 清单

Cubit/BLoC	对应页面	State freezed 字段
`AuthCubit`	登录/注销/改密	`user / token / role / mustChangePassword`
`DashboardCubit`	首页	`metrics / expiringContracts / overdueInvoices / unreadNotifications`
`NoiDetailCubit`	NOI 明细	`pgi / vacancyLoss / egi / opex / noi / noiMargin / breakdown`
`WaleDetailCubit`	WALE 明细	`overallWale / byType / contracts`
`KpiDashboardCubit`	KPI 看板	`totalScore / rank / metrics / rankings`
`AssetOverviewCubit`	资产总览	`buildings / typeStats`
`BuildingDetailCubit`	楼栋详情	`building / floors`
`FloorMapCubit`	楼层图	`svgUrl / units / selectedUnit`
`UnitDetailCubit`	房源详情	`unit / renovations`
`ContractListCubit`	合同列表	`contracts / meta / filters / hasMore`
`ContractDetailCubit`	合同详情	`contract / phases / deposits`
`FinanceOverviewCubit`	财务概览	`summary / overdueInvoices / roleData`
`InvoiceListCubit`	账单列表	`invoices / meta / filters / hasMore`
`WorkOrderListCubit`	工单列表	`orders / meta / filters / hasMore`
`WorkOrderDetailCubit`	工单详情	`order / logs / photos`
`WorkOrderFormCubit`	工单提报	`form / buildings / floors / units / contracts / issueTypeOptions`
`SubleaseListCubit`	二房东列表	`items / meta / hasMore`
`SubleaseDetailCubit`	二房东详情	`item / logs`
`NotificationCubit`	通知中心	`items / unreadCount / meta / activeTab / hasMore`
`ApprovalCubit`	审批队列	`items / meta / activeTab / hasMore`
`DunningCubit`	催收记录	`items / pendingCount / weekCount / overdueAmount / hasMore`

> 四态模式: 所有 State 为 `@freezed` sealed union: `initial` / `loading` / `loaded(data)` / `error(message)` 。

B.2 Admin Pinia Store 清单

> 与 v1.8 附录 B 完全一致，共 35+ Store。所有 Store 使用 `defineStore(id, setup)` setup 风格。

附录 C：v1.8 → v1.9 变更摘要

变更项	说明
移动端框架	uni-app (Vue 3 + wot-design-uni) → Flutter (Dart 3 + flutter_bloc + Material 3)
移动端导航	`pages.json` + `uni.navigateTo` → `go_router` + `StatefulShellRoute` + `NavigationBar`
状态管理	Pinia Store → BLoC/Cubit + `@freezed` 四态
依赖注入	— → `get_it` + `injectable`
组件体系	`wd-*` (wot-design-uni) → Material 3 Widget + 自定义 Widget
列表分页	`scroll-view` + `wd-loadmore` → `PaginatedListView` + `BlocBuilder`
筛选组件	`wd-drop-menu` / `wd-tag` → `FilterChipBar` / `SegmentedButton`
弹窗模式	`wd-popup` → `showModalBottomSheet`
表单	`wd-form` → `Form` + `TextFormField`
扫码	`uni.scanCode` → `mobile_scanner`
图片选择	`wd-upload` → `image_picker` + 自定义 `_PhotoPicker`
日期显示	`dayjs` → `DateFormat` (intl) `.format(dt.toLocal())`
色彩体系	`wd-tag type` → `ThemeExtension<CustomColors>` + `colorScheme.*`
平台覆盖	iOS / Android / 微信小程序 → iOS / Android / HarmonyOS Next
Admin 端	不变（Vue 3 + Element Plus + Pinia + axios）

文档结束。如有疑问或需进一步细化单个页面交互，请联系前端负责人。